



WEAPON s AMMUNITION MANAGEMENT SYSTEM

Data Structures C Algorithms





**DEPARTMENT OF ELECTRICAL
ENGINEERING, COLLEGE OF E&ME,
NUST, RAWALPINDI**

CS-250 DATA STRUCTURES & ALGORITHMS

Instructor: Dr Ishfaq Hussain

Lab Engineer: Moiz Shahid

SEMESTER PROJECT REPORT

Name	CMS ID	Dept/Syndicate
M.Fahad Ali	507319	EE-46-A
Muzammil	503292	EE-46-A
M. Bilal Chaudhry	510887	EE-46-A



ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my course instructor **Dr Ishfaq Hussain** and Lab Engineer **Sir Moiz Shahid** for the Data Structures and Algorithms subject for providing valuable guidance, encouragement, and support throughout the development of this project. Their continuous motivation and insightful suggestions played a crucial role in the successful completion of this work.

I am also thankful to my department and institution for providing the necessary resources and learning environment that helped me enhance my understanding of Data Structures and C++ programming.



Contents:

ABSTRACT	4
1. INTRODUCTION.....	5
2. OBJECTIVES OF THE PROJECT	5
3. SCOPE OF THE PROJECT	5
Included Features	5
4. DATA STRUCTURES USED.....	6
4.1 Singly Linked List	6
Usage:	6
Advantages.....	6
4.2 Recursion	7
Recursion is used for	7
Benefits	7
4.3 Binary Search Tree (BST)	7
Operations Implemented.....	7
Advantages.....	7
4.4 Sorting Algorithms.....	8
5. SYSTEM DESIGN	8
5.1 Classes and Structures.....	8
5.2 Menu-Driven Design	9
6. FUNCTIONAL DESCRIPTION.....	9
Menu Options.....	9
7. TIME COMPLEXITY ANALYSIS.....	14
Operation	14
Time Complexity.....	14
8. OUTPUT RESULTS	14
9. GUI IMPLENTATION	15
10. CONCLUSION	16
11. REFERENCES	16



ABSTRACT

This project presents an Inventory Management System(Weapon Ammunition Management) implemented using C++ and fundamental Data Structures and Algorithms (DSA). The system manages different categories of items such as handguns, rifles, armor, and ammunition. It supports operations like adding new items, updating quantities, searching items using recursion, sorting inventory using linked lists, and visualizing item IDs using a Binary Search Tree (BST).

A logging mechanism records all actions performed during the session and generates a final report upon exit. The project demonstrates the practical application of linked lists, recursion, sorting algorithms, and tree data structures in a real-world scenario.



1. INTRODUCTION

Inventory management is an essential component of many real-world systems such as warehouses, military storage, and retail stores. Efficient inventory handling requires fast searching, structured storage, and easy updates. Data Structures play a crucial role in optimizing these operations.

This project implements an inventory management system using C++ where dynamic data storage, searching, sorting, and hierarchical representation are achieved using appropriate data structures. The project is menu-driven and operates entirely through the console.

2. OBJECTIVES OF THE PROJECT

The main objectives of this project are:

- To design an inventory management system using C++
- To implement a dynamic inventory using linked lists
- To apply recursion for searching operations
- To implement sorting algorithms on linked list data
- To use Binary Search Tree (BST) for organized item ID storage
- To generate a session activity report using a logging system
- To demonstrate practical applications of Data Structures and Algorithms

3. SCOPE OF THE PROJECT

Included Features

- Adding new inventory items
- Updating item quantities



- Searching items by ID
- Sorting items by name and type
- Displaying inventory by categories
- Generating a Binary Search Tree of item IDs
- Generating a session report of performed actions

4. DATA STRUCTURES USED

4.1 Singly Linked List

The inventory and log system are implemented using singly linked lists.

Usage:

- Inventory items storage
- Logging system for session activities

Advantages:

- Dynamic memory allocation
- Efficient insertion operations
- No fixed size limitation

```
struct Item
{
    int id;
    string name;
    string type;
    int quantity;
    Item* next;
};

struct Log
{
    string message;
    Log* next;
};
```



4.2 Recursion

Recursion is used for:

- Searching items in the inventory
- BST insertion
- BST inorder traversal

Benefits:

- Simplified code structure
- Natural traversal of linked and tree structures

```
Item* searchRec(Item* node, int id)
```

4.3 Binary Search Tree (BST)

The BST is used to store item IDs and display them in sorted order.

Operations Implemented:

- Insert
- Inorder traversal

Advantages:

- Organized hierarchical data
- Faster searching compared to linear structures (average case)



```
Node* insertRec(Node* node, int key)
void inorder(Node* node)
```

```
struct Node
{
    int key;
    Node* left;
    Node* right;
};
```

4.4 Sorting Algorithms

A bubble-sort-like approach is used to sort linked list elements by:

- Item name
- Item type (using custom ranking)

```
void sortByName()    void sortByType()
```

5. SYSTEM DESIGN

5.1 Classes and Structures

- **Item** – Represents inventory items
- **Inventory** – Manages inventory operations
- **Procurement** – Handles item addition and updates
- **Log** – Stores system logs



- **BST** – Handles Binary Search Tree operations

5.2 Menu-Driven Design

The system operates using a menu-driven interface where the user selects options to perform specific actions. The program continues execution until the user selects the exit option.

```
void menu()  
switch(choice)
```

6. FUNCTIONAL DESCRIPTION

Menu Options

```
Menu:  
1.  Display Inventory  
2.  Add New Item  
3.  Remove Item  
4.  Update Item  
5.  Sort by Name  
6.  Sort by Type  
7.  Sort by ID  
8.  Search Item by ID  
9.  Display BST  
10. Exit
```

1. Display Inventory



```

Enter choice: 1
-----
HANDGUNS:
105      | Name: SIG P226      | Qty: 20
104      | Name: Beretta M9    | Qty: 44
103      | Name: DesertEagle   | Qty: 34
102      | Name: M1911         | Qty: 55
101      | Name: Glock17       | Qty: 10
-----
RIFLES:
205      | Name: Steyr AUG     | Qty: 30
204      | Name: FN Scar-L     | Qty: 15
203      | Name: HK416         | Qty: 12
202      | Name: M4A1          | Qty: 15
201      | Name: AK47          | Qty: 20
-----
ARMOR:
303      | Name: Kevlar Helmet | Qty: 24
302      | Name: SteelPlate    | Qty: 6
301      | Name: KevlarVest    | Qty: 8
-----
AMMO:
405      | Name: .50 BMG       | Qty: 295
404      | Name: 12 Gauge      | Qty: 300
403      | Name: 7.62x39mm     | Qty: 180
402      | Name: 5.56x45mm     | Qty: 150
401      | Name: 9mm           | Qty: 200
-----

```

2. Add New Item

```

Enter choice: 2
Enter (ID Name Type Qty): 206
M416
Rifle
100

```

3. Remove Item



```

Enter choice: 3
Enter ID to remove: 302
Enter quantity to remove: 20
Insufficient stock. Removed 6
Item auto-restocked to 50.

```

4. Update Item

```

Enter choice: 4
Enter ID: 103
Enter quantity to add: 25

```

5. Sort Inventory by name

```

Enter choice: 5
-----
HANDGUNS:
104 | Name: Beretta M9 | Qty: 44
103 | Name: DesertEagle | Qty: 59
101 | Name: Glock17 | Qty: 10
102 | Name: M1911 | Qty: 55
105 | Name: SIG P226 | Qty: 20
-----
RIFLES:
201 | Name: AK47 | Qty: 20
204 | Name: FN Scar-L | Qty: 15
203 | Name: HK416 | Qty: 12
206 | Name: M416 | Qty: 100
202 | Name: M4A1 | Qty: 15
205 | Name: Steyr AUG | Qty: 30
-----
ARMOR:
303 | Name: Kevlar Helmet | Qty: 24
301 | Name: KevlarVest | Qty: 8
302 | Name: SteelPlate | Qty: 50
-----
AMMO:
405 | Name: .50 BMG | Qty: 295
404 | Name: 12 Gauge | Qty: 300
402 | Name: 5.56x45mm | Qty: 150
403 | Name: 7.62x39mm | Qty: 180
401 | Name: 9mm | Qty: 200
-----

```

6. Sort Inventory by Type



```

Enter choice: 6
-----
HANDGUNS:
104 | Name: Beretta M9 | Qty: 44
103 | Name: DesertEagle | Qty: 59
101 | Name: Glock17 | Qty: 10
102 | Name: M1911 | Qty: 55
105 | Name: SIG P226 | Qty: 20
-----
RIFLES:
201 | Name: AK47 | Qty: 20
204 | Name: FN Scar-L | Qty: 15
203 | Name: HK416 | Qty: 12
206 | Name: M416 | Qty: 100
202 | Name: M4A1 | Qty: 15
205 | Name: Steyr AUG | Qty: 30
-----
ARMOR:
303 | Name: Kevlar Helmet | Qty: 24
301 | Name: KevlarVest | Qty: 8
302 | Name: SteelPlate | Qty: 50
-----
AMMO:
405 | Name: .50 BMG | Qty: 295
404 | Name: 12 Gauge | Qty: 300
402 | Name: 5.56x45mm | Qty: 150
403 | Name: 7.62x39mm | Qty: 180
401 | Name: 9mm | Qty: 200
-----

```

7. Sort Inventory by ID

```

Enter choice: 7
-----
HANDGUNS:
101 | Name: Glock17 | Qty: 10
102 | Name: M1911 | Qty: 55
103 | Name: DesertEagle | Qty: 59
104 | Name: Beretta M9 | Qty: 44
105 | Name: SIG P226 | Qty: 20
-----
RIFLES:
201 | Name: AK47 | Qty: 20
202 | Name: M4A1 | Qty: 15
203 | Name: HK416 | Qty: 12
204 | Name: FN Scar-L | Qty: 15
205 | Name: Steyr AUG | Qty: 30
206 | Name: M416 | Qty: 100
-----
ARMOR:
301 | Name: KevlarVest | Qty: 8
302 | Name: SteelPlate | Qty: 50
303 | Name: Kevlar Helmet | Qty: 24
-----
AMMO:
401 | Name: 9mm | Qty: 200
402 | Name: 5.56x45mm | Qty: 150
403 | Name: 7.62x39mm | Qty: 180
404 | Name: 12 Gauge | Qty: 300
405 | Name: .50 BMG | Qty: 295
-----

```



8. Search Item by ID

```
Enter choice: 8
Enter ID: 405
.50 BMG | Ammo | Qty: 295
```

9. Display Binary Search Tree

```
Enter choice: 9
BST (Inorder Traversal): 101 102 103 104 105 201 202 203 204 205 206 301 302 303 401 402 403 404 405
```

10. Exit and Generate Session Report

```
Enter choice: 10
Exiting the Program.
=====
SESSION REPORT
Actions performed during this session:
- Added item: Glock17 (Handgun)
- Added item: M1911 (Handgun)
- Added item: DesertEagle (Handgun)
- Added item: Beretta M9 (Handgun)
- Added item: SIG P226 (Handgun)
- Added item: AK47 (Rifle)
- Added item: M4A1 (Rifle)
- Added item: HK416 (Rifle)
- Added item: FN Scar-L (Rifle)
- Added item: Steyr AUG (Rifle)
- Added item: KevlarVest (Armor)
- Added item: SteelPlate (Armor)
- Added item: Kevlar Helmet (Armor)
- Added item: 9mm (Ammo)
- Added item: 5.56x45mm (Ammo)
- Added item: 7.62x39mm (Ammo)
- Added item: 12 Gauge (Ammo)
- Added item: .50 BMG (Ammo)
- Displayed inventory
- Added item: M416 (Rifle)
- Procurement added: M416
- Searched item ID 302 (FOUND)
- Insufficient stock for ID 302. Removed remaining 6
- Auto-restocked ID 302 to quantity 50
- Searched item ID 103 (FOUND)
- Updated item ID 103 qty +25
- Displayed inventory
- Sorted inventory by NAME
- Displayed inventory
- Sorted inventory by TYPE
- Displayed inventory
- Sorted inventory by ID
- Displayed inventory
- Displayed BST
- Searched item ID 405 (FOUND)
```

Each option executes a specific function and logs the performed action.



7. TIME COMPLEXITY ANALYSIS

Operation	Time Complexity
Add Item	$O(1)$
Search Item (Linked List)	$O(n)$
Sort Inventory	$O(n^2)$
BST Insertion	$O(\log n)$ (Average)
BST Traversal	$O(n)$

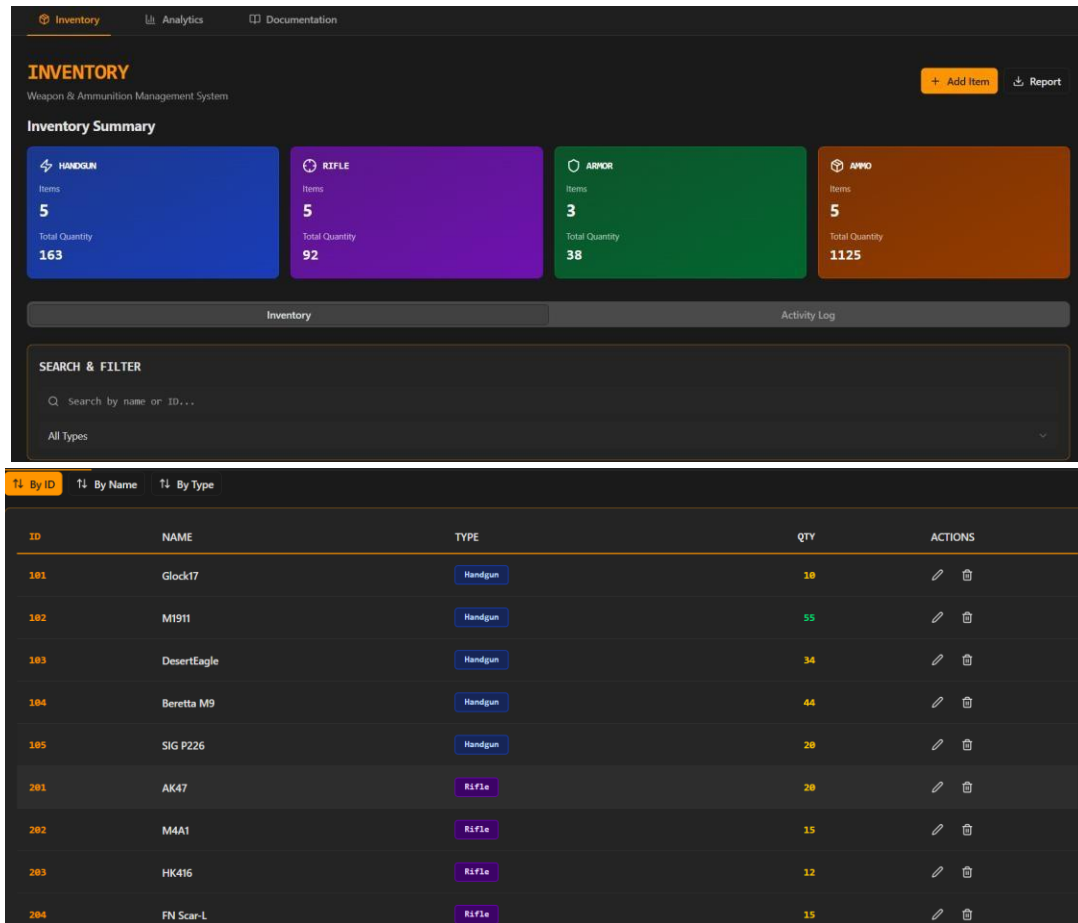
8. OUTPUT RESULTS

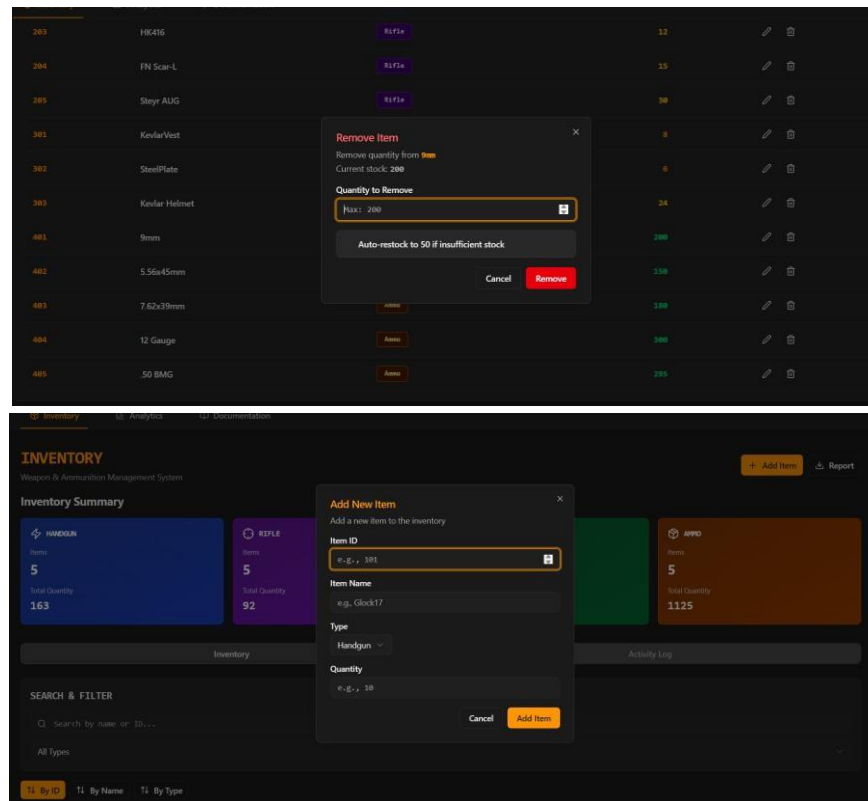
The system successfully:

- Displays categorized inventory
- Searches items using recursive logic
- Sorts items correctly
- Displays sorted item IDs using BST inorder traversal
- Generates a detailed session activity report upon exit



9. GUI IMPLEMENTATION





10. CONCLUSION

This project successfully demonstrates the use of Data Structures such as Linked Lists, Recursion, Sorting Algorithms, and Binary Search Trees in building an efficient inventory management system. The implementation improved understanding of how DSA concepts are applied in real-world applications. The project highlights the importance of data organization, efficient searching, and structured programming in software development.

11. REFERENCES

1. Data Structures and Algorithms – Lecture Notes
2. *C++ Programming Language* by Bjarne Stroustrup
3. www.cppreference.com

